

Linguagens Formais e Compiladores: Processo de Compilação

Profa. Jerusa Marchi

Universidade Federal de Santa Catarina
Departamento de Informática e Estatística

2023-1

Estrutura Geral do Compilador

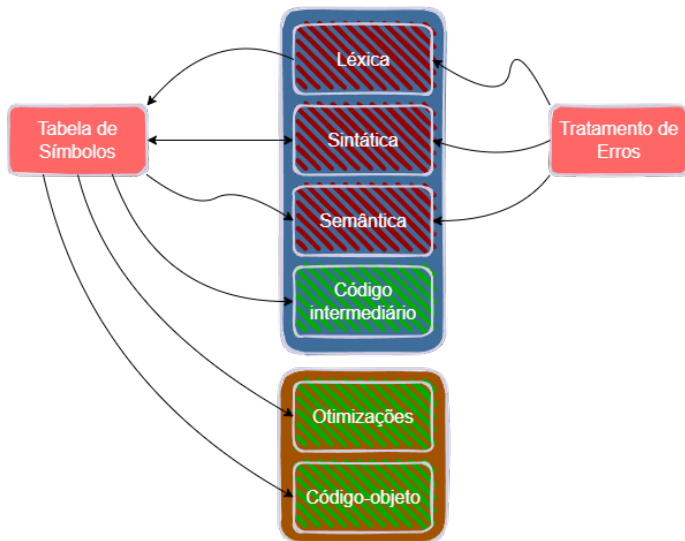


Tabela de Símbolos

- Estrutura onde são armazenadas as informações essenciais (**atributos**) de cada identificador utilizado no programa fonte
- Atributos mais comuns:
 - ▶ nome
 - ▶ endereço relativo (nível e deslocamento)
 - ▶ categoria:
 - ★ variável (simples: tipo; array: dimensões, tipo dos elementos; registro: campos)
 - ★ constante (tipo e valor)
 - ★ procedimento (procedimento ou função; número de parâmetros; ponteiro para os parâmetros; se função, o tipo do retorno)
 - ★ parâmetro (tipo; forma de passagem - valor/referência)
 - ★ campo de registro (tipo, deslocamento dentro do registro)

Tratamento de Erros

Funções:

- diagnosticar erros léxicos, sintáticos e semânticos encontrados na etapa de análise
 - ▶ dado os aspectos formais dos mecanismos de análise, o diagnóstico de erro é preciso
- tratar erros de forma que a análise possa prosseguir
 - ▶ implementar técnicas de recuperação de erros
 - ★ ignorar o erro e prosseguir a partir da próxima construção sintática
 - ★ abortar o processo de compilação no primeiro erro encontrado

Analizador Léxico

- Também chamado de *scanner*
- Interface entre o programa fonte e o compilador
- Funções:
 - ▶ ler o programa fonte
 - ▶ agrupar caracteres em sequências significativas - *lexemas*
 - ▶ produz como saída uma sequência de *tokens* - par $\langle \textit{nome} - \textit{token}, \textit{valor} - \textit{atributo} \rangle$
 - ★ *nome* - *token* - identifica o tipo de item léxico encontrado (identificadores, palavras reservadas, constantes, símbolos especiais)
 - ▶ ignorar elementos sem valor sintático (espaços em branco, comentários e caracteres de controle)
 - ▶ detectar erros léxicos (símbolos inválidos)

Analizador Léxico

Exemplo 1:

`montante = dep_inicial + t_juros * 60`

- `montante` é um **lexema** mapeado para o **token** $\langle id, 1 \rangle$ onde *id* é um símbolo abstrado que significa identificador e 1 aponta para a entrada da tabela de símbolos (TS) onde se encontra `montante`
- O símbolo de atribuição `=` é mapeado para $\langle = \rangle$ (sem valor de atributo)
- `dep_inicial` é mapeado para o token $\langle id, 2 \rangle$ onde 2 é onde se encontra `dep_inicial` na TS
- `+` é mapeado para $\langle + \rangle$
- `t_juros` é mapeado para $\langle id, 3 \rangle$ onde 3 aponta `t_juros` na TS
- `*` é um lexema mapeado para o token $\langle * \rangle$
- `60` é mapeado para $\langle num, 4 \rangle$, sendo o valor 60 armazenado na TS

Analizador Léxico

Exemplo 2:

Programa Fonte	Tokens Reconhecidos	
program exemplo;	program	1 - PR
var A, B : integer;	exemplo	2 - ID
begin	;	3 - SE
(*Inicio do programa*)	var	4 - PR
read (A);	A	5 - ID
B := A + 2.5;	,	6 - SE
...	...	...
end.	end	35 - PR
	.	36 - SE

Analizador Léxico

A fase de análise é terminada ainda que erros léxicos sejam identificados, contudo não faz sentido prosseguir na análise sintática

Analizador Léxico

Implementação de analisadores léxicos:

- Geradores Automáticos de Analisadores Léxicos (LEX)
- Entrada: especificação léxica da linguagem
- Formalismo: Gramáticas Regulares e Autômatos Finitos

Analizador Sintático

Funções:

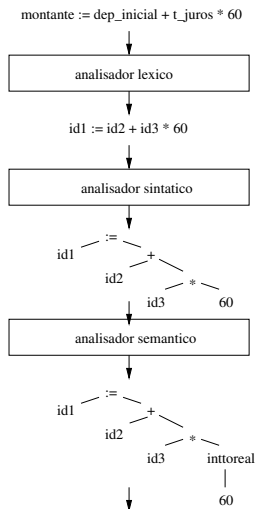
- agrupar os **tokens** em estruturas sintáticas (expressões, comandos, declarações, etc.) na forma de uma árvore de derivação
- verificar se a sintaxe da linguagem na qual o programa foi escrito está sendo respeitada
 - ▶ detectar/diagnosticar erros sintáticos - lexema que desrespeita a estrutura sintática esperada

Analizador Sintático

Exemplo:

tabela de simbolos

1	montante	...
2	dep_inicial	...
3	t_juros	...
4		



Analizador Sintático

Implementação de analisadores sintáticos:

- Geradores Automáticos de Analisadores Sintáticos (YACC)
- Entrada: especificação sintática da linguagem (gramática)
- Formalismo: Gramáticas Livres de Contexto e Autômatos de Pilha

Analizador Semântico

Utiliza a árvore sintática e as informações na tabela de símbolos para verificar a consistência semântica do programa fonte com a definição da linguagem

- verificação da compatibilidade de tipos (coerção), declaração de variáveis (ausência ou múltiplas declarações), coerência entre declaração e uso de identificadores

Reúne informações sobre os tipos e as salva na árvore sintática (árvore sintática anotada) para uso na geração de código intermediário

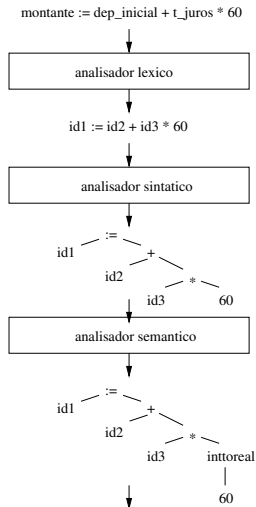
Detecta erros semânticos

Analizador Semântico

Exemplo 1:

tabela de simbolos

1	montante	...
2	dep_inicial	...
3	t_juros	...
4		



Analizador Semântico

Exemplo 2:

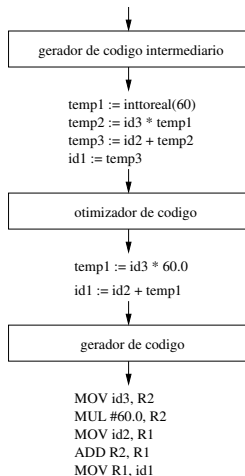
Var B: array [1..10] of real;	Var A: string[30];
...	B: integer;
B:= X;	...
...	B := B + A;
	...

Gerador de código intermediário

- Consiste na geração de um conjunto de instruções equivalentes ao programa fonte para uma máquina hipotética (virtual)
- A representação intermediária deve ser facilmente produzida e ser facilmente traduzida para a máquina alvo
- Objetivos:
 - ▶ facilitar a tradução para a linguagem objeto
 - ▶ permitir a otimização de código
 - ▶ portabilidade

Gerador de código intermediário

Exemplo:

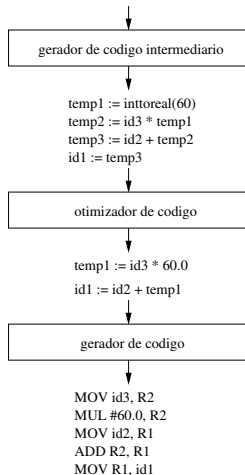


Otimizador de Código

- Melhorar o código, de forma que a execução seja mais eficiente quanto ao tempo e/ou espaço ocupado
- Otimizações mais frequentes:
 - ▶ agrupamentos de sub-expressões comuns
 - ▶ retirada de comandos invariantes ao LOOP
 - ▶ eliminação de código inalcançável
 - ▶ alocação ótima de registradores
 - ▶ Redução em força (linearização de código)
- Compromisso entre tempo de compilação (gasto em otimização) $\times$ tempo de vida do código, número de execuções do código

Otimizador de Código

Exemplo:



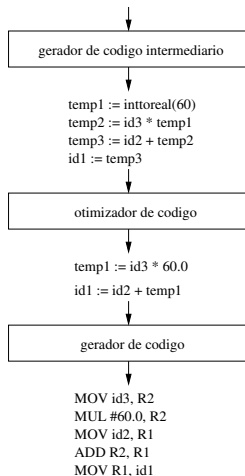
Gerador de Código Objeto

Converte o programa fonte (a partir de sua representação em código intermediário) para uma sequência de instruções (assembly ou máquina) de uma máquina real

- uma nova otimização de código pode ser executada sobre o código final - otimização de código dependente da máquina

Gerador de Código Objeto

Exemplo:



Gerador de código intermediário

Exemplo 2: $E := (A + B) * (C + D)$

Quádrupla	Máquina de acumulador
(+, A, B, T1)	carregue A
(+, C, D, T2)	some B
(*, T1, T2, E)	armazena T1
	carregue C
	some D
	armazene T2
	carregue T1
	multiplique T2
	armazene E

Bibliografia

- AHO, A.V.; LAM, M.S.; SETHI, R. ULLMAN, J.D., **Compiladores - Princípios, Técnicas e Ferramentas**, 2ª edição, Ed. Addison Wesley, 2008 (capítulo 1 - Introdução)